Project Case Study

Beginner-friendly case study of how one STM32 project can evolve with this workflow, including prompts, tradeoffs, and host-tool growth.

Beginner-friendly case study of how one STM32 project can evolve with this workflow, including prompts, tradeoffs, and host-tool growth.

This document is not a rulebook. It is a taste of what work in this STM32 workflow can look like when a project grows from a simple board bring-up into a more complete firmware and host-tool system.

It is written as a beginner-facing example on purpose:

- it shows the kind of prompts a user might give
- it shows the kinds of changes the workflow can help organize
- it focuses on project development rather than starter-kit maintenance

> **Important**
> Treat this as an example journey, not as a required sequence. Real projects will take different paths depending on the board, peripherals, and product goals.

At A Glance

The fastest way to understand this example project is to look at the main development steps first.

This is the shape of the project the workflow helped build:

1. initialize a reproducible STM32 project from the workflow
2. implement interrupt-driven UART communication
3. add optional debug output over the same serial path without breaking normal control replies
4. implement STEP/DIR stepper-motor control using MCU timers
5. implement analog signal acquisition with ADC + DMA buffering + averaging

6. build a text-based serial control interface for motion, status, and config
7. port the runtime to a multitasking model with FreeRTOS
8. implement the USB device interface
9. implement a binary USB control protocol that coexists with the text UART interface
0. implement a C# graphical host application for motor control and monitoring

The exact order overlapped in places, and real projects often move back and forth between these steps. But as a first impression, this list shows what the project is about much faster than the detailed story does.

What This Example Is Trying To Show

A new user usually does not need a full theory of the workflow first. What they often want is a feeling for questions like:

- what would I actually ask the agent to do
- what kinds of project steps fit this workflow well
- how does the work move from `.ioc` and generated code into real firmware
- what does "build, flash, validate, and keep evidence" feel like in practice

This example answers those questions by showing one realistic project path.

The General Pattern

In this workflow, project work often looks like this:

1. describe the next concrete goal in plain language
2. let the agent inspect the repo, config, and generated boundaries
3. implement one project-facing step
4. validate honestly
5. keep the repo state, evidence, and docs coherent

That pattern is useful for much more than just "build the firmware." It also helps with:

- restructuring code safely

- keeping `.ioc` and code generation aligned
- adding peripherals such as timers, ADC, USB, and RTOS
- designing control and debug interfaces
- growing into host-side tools and UI work

Phase 1: Start From A Reproducible Base

The project begins with a starter workflow and a real target board. The first goal is not polish. The first goal is to get to a clean, reproducible baseline.

Example prompt:

> "Check the toolchain, config, and board readiness. Then set up a minimal STM32 project structure from the current `.ioc` and tell me what is missing."

Typical outcome:

- toolchain and config are checked first
- the project gets a repeatable build path
- the `.ioc` becomes the source of truth for generated configuration
- the repo starts collecting evidence instead of relying on memory

Why this matters:

- the project starts from something that can be rebuilt and checked later
- environment problems are discovered early instead of after a lot of coding

Phase 2: Turn Generated Code Into A Real Firmware Structure

A generated CubeMX project is only a starting shell. Very quickly, the project needs a real internal architecture.

Example prompt:

> "Refactor this STM32 project into a clean application structure. Keep `main.c` minimal, move feature logic into modules, and preserve the generated boundaries so CubeMX regeneration stays manageable."

Typical project changes:

- keep `main.c` as orchestration only
- add modules such as:
- `app`
- `stepper`
- `motor_protocol`
- `a2d`
- `uart_console`
- route HAL callbacks through a controlled app layer
- keep generated code and project code from mixing chaotically

Why this matters:

- later feature work becomes easier
- CubeMX regeneration is less dangerous
- review becomes more readable

Phase 3: Add The Stepper Control Core

Once the structure exists, the real product behavior starts to appear. For this example project, that meant building the motion-control core.

Example prompt:

> "Implement a stepper-control core with enable, move, jog, stop, and zero operations. Use HAL timers, keep the motion logic in its own module, and make the results easy to validate from a command interface."

Typical project changes:

- add timer-based pulse generation

- add a control timer for motion updates
- support practical motion operations:
- enable/disable
- relative move
- absolute move
- jog
- stop
- zero
- keep stepper behavior behind a dedicated module instead of scattering it through generated files

What the workflow contributes here:

- it encourages small, testable steps
- it makes it natural to rebuild, flash, and validate after each capability
- it keeps the codebase from collapsing into one giant `main.c`

> **Hypothetical Discussion Example**
> This is the kind of back-and-forth a user might have with the agent while deciding how stepper pulses should be generated.
>
> **Agent:** We could count pulses in an interrupt after each pulse event, or we could rely more heavily on hardware timers and counters to reduce CPU load.
>
> **User:** Hardware counting is efficient, but interrupt-driven updates give us more freedom for acceleration, deceleration, and custom motion behavior.
>
> **Agent:** That tradeoff is real. A hardware-centered path is attractive when low CPU overhead and simple motion behavior are the main goals. An interrupt-driven path is better when the motion profile needs tighter software control.
>
> **User:** If both are reasonable, we should not force a false either/or decision too early.
>
> **Agent:** Then a practical design is to implement both modes and expose the choice as a project configuration option.
>
> **User:** Could there also be a third option where hardware timing is combined with DMA-fed table data so each pulse update loads the next timer value from memory and follows an arbitrary precomputed profile?

> **Agent:** Yes. A timer/DMA table-driven mode is also possible and can support rich motion profiles with very low CPU load, but it adds setup complexity and is usually best when the profile is known in advance.
>
> **Final implementation focus:** for this project stage, implement the first two modes, keep the tradeoff explicit, and leave the timer-plus-DMA-table option documented as a valuable future path rather than building all three at once.

Why this discussion is worth showing:

- it demonstrates that the workflow can handle real design tradeoffs, not only straightforward coding tasks
- it shows that the agent can help compare implementation strategies instead of only writing whichever one was mentioned first
- it shows that additional options can be surfaced and deferred deliberately rather than forgotten
- it gives the reader another example of how to challenge the agent until the design matches the project's actual priorities

Phase 4: Add ADC And DMA Without Losing Coherence

Many real projects need measurement, not just output control. This example project added ADC acquisition as part of the runtime picture.

Example prompt:

> "Add ADC1 DMA circular acquisition for analog sampling. Keep the callbacks safe, keep the `.ioc` and IRQ/DMA setup coherent, and expose the measured value through the project status path."

Typical project changes:

- enable ADC in `.ioc`
- configure DMA circular buffering
- handle half-transfer and transfer-complete callbacks
- compute stable project-facing values from the DMA buffer

- keep the IRQ and DMA behavior explicit instead of hidden in ad-hoc code

Why this phase is a good workflow example:

- it shows the value of treating `.ioc` as the hardware source of truth
- it shows how the workflow can help keep DMA, NVIC, and firmware code aligned
- it demonstrates a common STM32 pattern that is easy to get subtly wrong

> **Hypothetical Discussion Example**
> This is the kind of timing discussion a user might have with the agent when a DMA-based ADC design is technically possible but easy to reason about incorrectly.
>
> **Agent:** We should estimate the conversion timing, the full-buffer timing, and the time available before the oldest circular-buffer samples are overwritten.
>
> **Agent:** A single circular DMA buffer with only a transfer-complete interrupt can be risky. Safer alternatives include stopping DMA while processing, using half-transfer plus transfer-complete callbacks, or using a true double buffer.
>
> **User:** The total interrupt duration is not the real question. What matters is whether the ISR can read the first element before it is overwritten, then keep reading each later element before its own overwrite time arrives.
>
> **Agent:** That objection is valid. The correct criterion is not "ISR time must be shorter than one conversion." The correct criterion is whether the processing pace stays ahead of the DMA overwrite pace for each element that still belongs to the completed cycle.
>
> **User:** Then if the timing margin is sufficient, there is no need to stop DMA or add a second buffer just for safety theater.
>
> **Agent:** Agreed. A single circular buffer with transfer-complete processing can be valid, but it should be treated as a timing-proven design rather than as a universal assumption. Half-transfer or double buffer still provide stronger margin when rates or worst-case latency become tighter.
>
> **Final implementation focus:** keep the ADC acquisition design explicit, use a DMA-based buffered approach with coherent callback handling, and choose the final single-buffer vs stronger-margin variant based on measured timing needs rather than vague intuition.

Why this discussion is worth showing:

- it demonstrates that a good user prompt can challenge the agent's reasoning
- it shows that disagreement can improve the final engineering conclusion
- it gives the reader a model for how to ask follow-up questions when timing, concurrency, or coherence details matter

Phase 5: Build A Human Control Interface First

Before a polished UI exists, a project still needs a practical way to operate the board. For many STM32 projects, a UART text interface is the fastest useful front door.

Example prompt:

> "Create a human-friendly UART command interface for bring-up and service work. Add `help`, `status`, motion commands, config commands, and clear success/error replies."

Important framing:

- this interface is part of the firmware the agent helped design and implement
- the end user of the device is not expected to invent this command set by hand
- the prompt asks for the kind of interface needed, and the workflow helps turn that into a concrete, testable command surface

Typical project changes:

- add a UART console module
- support line-based text commands
- return clear `ok ...` and `err ...` responses
- make commands readable enough for terminal-based bring-up
- improve help output as the feature set grows

Agent-suggested resulting `help` response:

```
> help
ok help topics
```

```
topic: topics - list available help topics
topic: cmds - list all commands
topic: cmd <name> - show one command
topic: cfg - show configuration items
topic: units - show unit conversion rules
topic: err - show error codes
topic: examples - show examples
ok help done
```

Agent-suggested resulting command-list response:

```
> help cmds
ok help cmds
cmd: ping - reply with protocol/version
cmd: status - report motor and adc status
cmd: en <0|1> - set enable output
cmd: stop [soft|hard] - stop motion
cmd: zero - set current position to zero
cmd: move <distance_steps> <speed_steps_s> [accel_steps_s2] - relative
move in steps
cmd: goto <abs_pos_steps> <speed_steps_s> [accel_steps_s2] - absolute
move in steps
cmd: jog <speed_steps_s> [accel_steps_s2] - continuous move in steps/s
cmd: cfg <item> <value> - set steps_per_rev, steps_per_mm,
counter_mode, accel, decel
cmd: dbg [0|1|status] - set or query deferred debug logging
cmd: help [topics|cmds|cmd <name>|cfg|units|err|examples] - show help
ok help done
```

What those examples show:

- they show that the interface becomes self-describing
- they reduce the amount of external documentation needed during bring-up
- they make it clear that the AI-assisted workflow can help shape operator ergonomics, not only low-level driver code

Why the interface matters:

- it gives the user a fast manual test surface
- it reduces bring-up friction before a richer UI exists

- it turns hardware validation into something practical

Additional useful capability in this project:

- the same serial interface also became an agent-facing runtime path
- the agent could use it to help confirm that new firmware was really running
- it could query live board state during validation
- it could benefit from temporary debug output inserted at useful points and then analyze the resulting board responses
- the general idea, giving the agent a controlled way to communicate directly with the target board, can also be a very useful practice in other projects

Why the agent-facing path matters:

- it shows that the interface can support not only manual operator work but also agent-assisted runtime validation
- it gives the workflow a practical way to observe real board behavior after flashing, instead of stopping at build success alone
- it makes temporary instrumentation and response analysis more useful during bring-up and troubleshooting

Phase 6: Add Debug Visibility Without Breaking The Control Path

As projects grow, the need for debug output grows too. But casual debug prints often break timing, interleave messages badly, or fight with the main command channel.

Example prompt:

> "Add deferred debug logging over the shared UART without breaking protocol output. Keep it safe for task and ISR producers, and make debug enablement a runtime command instead of a permanent flood."

Typical project changes:

- add a dedicated logger task

- keep UART ownership single-writer
- allow non-blocking debug production from task and ISR context
- prefix debug lines clearly
- expose debug control through commands such as:
- `dbg`
- `dbg status`
- `dbg 1`
- `dbg 0`

Why this is a good workflow showcase:

- it shows that the workflow is not only about build automation
- it also supports disciplined runtime observability work

Phase 7: Grow Into RTOS And More Robust Runtime Behavior

A small project may begin with a simple loop, but it often needs a better runtime model as features accumulate.

Example prompt:

> "Move this project toward a cleaner RTOS-based runtime. Keep CubeMX-generated ownership where appropriate, move application logic behind module boundaries, and keep the result compatible with regeneration."

Typical project changes:

- integrate FreeRTOS and CMSIS-RTOS2
- move app flow into scheduled tasks
- replace blocking transmit paths with queued worker tasks
- keep task ownership boundaries clear
- keep `.ioc` task metadata and runtime code aligned

Why this matters:

- the project becomes more scalable
- concurrency decisions become more explicit
- generated RTOS scaffolding and project-owned behavior can coexist more safely

Phase 8: Keep CubeMX Generation Compatible With Real Development

A very practical part of STM32 work is dealing with regeneration. Users often adjust `.ioc`, regenerate, and then worry about what got damaged.

Example prompt:

> "I changed the `.ioc` in CubeMX and regenerated code manually. Reconcile the project, restore workflow-owned USER CODE boundaries, and verify that the generated tree still matches the intended architecture."

Typical project changes:

- regenerate from `.ioc`
- reconcile workflow-owned USER CODE areas
- restore project trampolines or cleanup steps when needed
- rerun checks before claiming the generated tree is aligned

Why this phase matters:

- it shows a realistic STM32 pain point
- it demonstrates that the workflow is meant to survive real CubeMX use, not just idealized greenfield coding

Phase 9: Add A Machine-Facing Control Interface

After the human-facing CLI is useful, a project may need a cleaner path for host software. In this example, the project added a binary USB control plane while keeping the UART text interface for humans.

Example prompt:

> "Keep UART as the human-facing CLI, but add a structured USB protocol for host tools and UI work. Use framed messages, explicit errors, and ownership rules so the two control paths do not fight each other."

Typical project changes:

- add a framed USB protocol
- define message ids, payloads, and result codes
- expose motion, status, config, and ownership commands
- keep the human and machine control surfaces separate
- add runtime ownership rules when multiple front doors exist

Why this is a strong example:

- it shows that the workflow can support system design, not just firmware files
- it demonstrates protocol evolution and interface discipline

Phase 10: Add Host Tools And An Operator UI

Once the machine-facing protocol exists, the project can grow beyond the board itself.

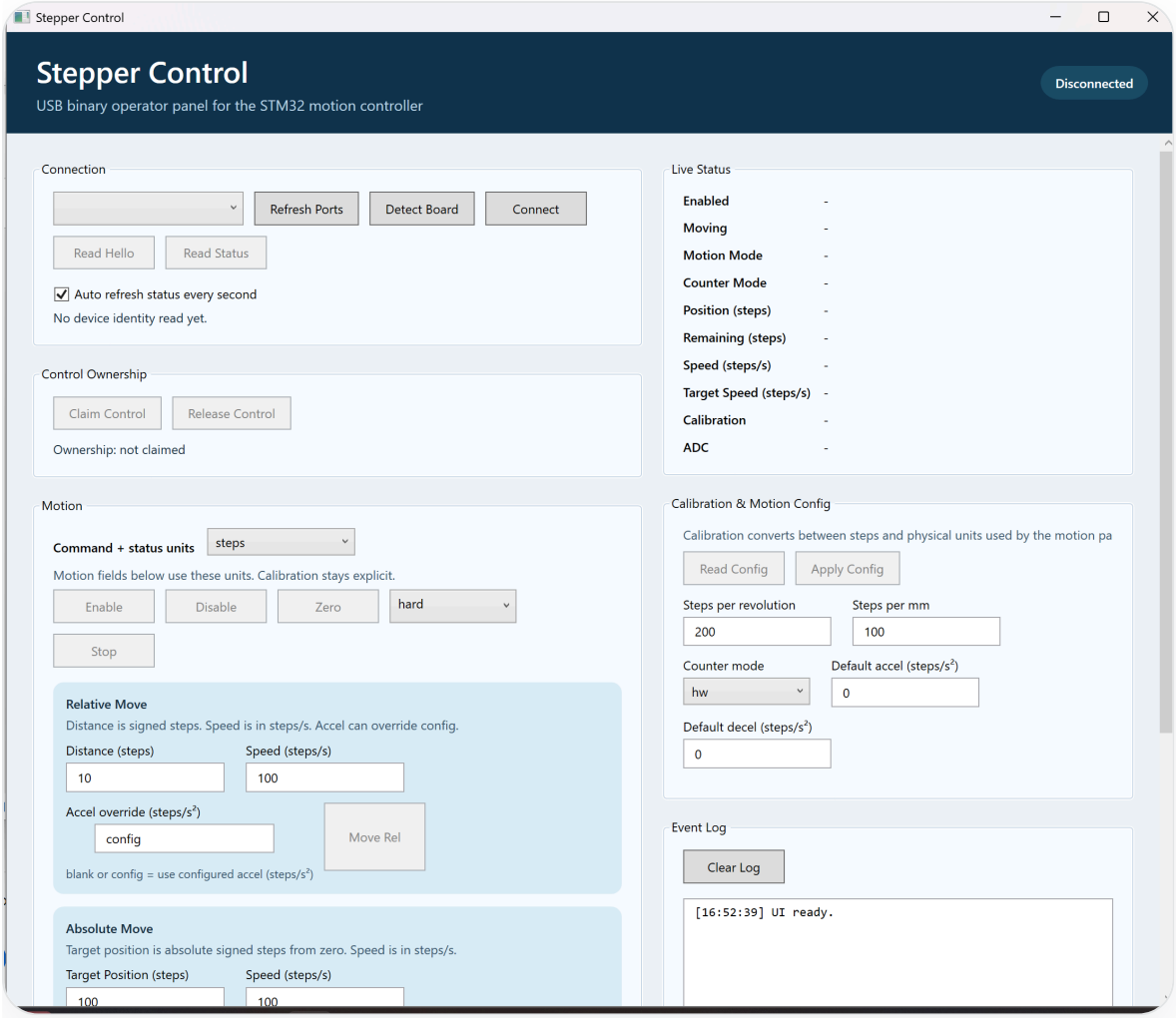
Example prompt:

> "Build host-side tools around the device protocol: start with a probe tool, then add a small Windows UI for connect, status, motion control, config, and event logging."

Typical project changes:

- add a reusable host-side protocol library
- add a console probe tool
- add an operator UI
- validate the host path against the real device protocol

One example UI result from this kind of workflow-guided project:



Caption: One later project outcome was a Windows operator panel for USB-based motion control, live status, calibration/config updates, ownership handling, and event monitoring.

Why this matters:

- the workflow now covers more than embedded code alone
- the project becomes a firmware-plus-host-tool system

What A Beginner Should Notice

This example is not impressive because it is large. It is useful because the work stays understandable while it grows.

The recurring themes are:

- plain-language prompts are enough to drive meaningful work

- `.ioc` stays important throughout the project
- generated code and project code are kept in a workable relationship
- validation is repeated after meaningful steps
- evidence matters
- interfaces are designed deliberately instead of emerging as accidents

A Small Set Of Example Prompts

If you want the shortest possible taste of how future work may feel, prompts like these are representative:

> "Check whether the toolchain, workflow config, and board connection are healthy."

> "Create a clean firmware structure from this `.ioc`. Keep `main.c` minimal and move feature logic into modules."

> "Implement timer-based stepper control with enable, move, jog, stop, and zero, then build and validate what you can."

> "Add ADC DMA circular sampling and expose a stable measurement through the status path."

> "Make the UART control interface more usable for bring-up and debugging."

> "Add deferred debug logging without breaking the normal command responses."

> "Reconcile the project after manual CubeMX generation and verify that the generated tree is still aligned."

> "Add a machine-facing USB protocol for host tools while keeping UART for humans."

> "Create a first host-side probe tool and operator UI around the device protocol."

Final Impression

The main promise of this workflow is not that it removes engineering work. The promise is that it helps keep the work organized, reproducible, and easier to validate as the project gets more ambitious.

That is the impression this example is meant to leave:

- you can start small
- you can add real embedded features gradually
- you can grow into interfaces, protocols, and host tools
- and the workflow can help keep the journey coherent